IMT Nord Europe
École Mines-Télécom
IMT-Université de Lille

Projet IoT — TempHumiX

Khadija Ouanour

UV DEVIOT · Année universitaire 2026–2027

Novembre 2026

Table des matières

Introduction générale 4
1. Simulation du capteur sous SimulIDE 5
1.1 Objectif et préparation 5
1.1.1 Organisation du projet 5
1.1.2 Compilation et exécution 5
1.1.3 Suivi avec Git 5
1.2 Prise en main des sorties numériques 5
1.2.1 Premier circuit avec une LED 5
1.2.2 Chenillard à dix LED 6
1.3 Lecture du DHT22 et test du seuil 6
1.4 Affichage des mesures sur le LCD 7
1.4.1 Essai du LCD seul 7
1.4.2 Association du LCD et du DHT22 8
1.5 Montage complet de TempHumiX 8
1.6 Programme d'acquisition 9
1.6.1 Gestion du temps dans la boucle principale 9
1.6.2 Dateur logiciel 10
1.6.3 Affichage et transmission 10
1.7 Vérification de la liaison série et bilan 10
1.7.1 Adaptation du test COM8 10
1.7.2 Bilan de la simulation 11
2. Communication Wi-Fi et protocole MQTT 12
2.1 Objectif et environnement 12
2.2 Connexion et premiers échanges avec MQTTX (5.2) 12
2.2.1 Questions de la partie 5.2.4 12
2.3 Programmation de la carte (6.1.1 à 6.1.3) 12
2.3.1 Acquisition, LCD et RTC 12
2.3.2 Connexion Wi-Fi et synchronisation NTP 13
2.4 Publication et commandes MQTT (6.1.4 et 6.1.5) 14
2.5 Réduction des publications (6.1.6) 15
2.6 Analyse du trafic avec Wireshark (6.2.1) 15
2.6.1 Réponses aux questions Wireshark 16
2.7 QoS, Retain et abonnements génériques (6.2.2) 17
2.8 Questions générales — pages 15 et 16 18
2.8.1 Acheminement et modèle OSI / TCP-IP 18
2.8.2 Services du routeur et place dans l'IoT 18
2.8.3 Microcontrôleur ou microprocesseur ? 18
2.8.4 Nombre de connexions Wi-Fi, risques et topologie 18
2.8.5 Sécurité du RUTX50 19
2.8.6 Analyse des logs Mosquitto 19
2.9 Bilan du TP MQTT 19
3. Interface de supervision sous PySide6 20
3.1 Objectif et environnement 20
3.1.1 Matériel et outils 20
3.2 Communication MQTT en mode console 20
3.2.1 Client MQTT en lecture (sections 3.2 et 3.3) 20
3.3 Envoi des commandes au capteur 21
3.3.1 Client MQTT en écriture (section 3.4) 21

3.4 Exécution parallèle et thread MQTT 22
3.4.1 Deux clients indépendants (section 3.5) 22
3.4.2 Réunion dans un même programme (section 3.6) 23
3.5 Interface graphique sous PySide6 23
3.5.1 Conception et conversion (sections 4.3 et 4.4) 23
3.5.2 Fonctionnement de l'application (section 4.5) 24
3.6 Notice de fonctionnement de l'IHM 24
3.7 Archivage des mesures dans SQLite 24
3.8 Consultation de l'historique sous forme de courbes 25
3.9 Bilan de l'IHM 25
Conclusion générale 26

Introduction générale

Le projet TempHumiX a été mené en trois étapes complémentaires. La première consiste à simuler les circuits autour d'une Arduino UNO R3, à lire les mesures d'un DHT22 et à les afficher sur un écran LCD. La deuxième transpose le montage sur une Arduino UNO R4 WiFi, avec des échanges MQTT entre le capteur et un poste de travail. La troisième ajoute une application Python pour commander le capteur, enregistrer les mesures et consulter leur historique.

Ce rapport rassemble les trois comptes rendus dans leur ordre de réalisation. Les captures illustrent les tests effectués ; les réponses aux questions du sujet sont conservées dans les sections correspondantes.

1. Simulation du capteur sous SimulIDE

1.1 Objectif et préparation

Le but du TP est de réaliser dans SimulIDE un capteur de température et d'humidité autour d'une carte Arduino UNO R3. J'ai testé séparément la commande des LED, la lecture du DHT22 et l'affichage I2C. J'ai ensuite réuni ces éléments dans TempHumiX pour afficher les mesures avec la date et l'heure et les transmettre sur une liaison série.

Cette première étape porte sur la simulation et la validation des différents circuits. Elle prépare le passage à la carte réelle et à la communication MQTT.

1.1.1 Organisation du projet

Les circuits et les programmes sont rangés dans le dossier SIMULIDE du projet. Pour chaque exercice, le sous-dossier, le circuit .sim1 et le croquis .ino portent le même nom de base. Cette convention permet de retrouver le code qui correspond à chaque circuit.

Dossier	Manipulation
clignotement_led	Clignotement d'une LED
bargraphe	Chenillard de dix LED
capteur_dht22	Commande de deux LED à partir d'un seuil de température
lcd_i2c	Affichage de texte sur le LCD
lcd_i2c_dht22	Affichage de la température et de l'humidité
TempHumiX	Montage complet avec horloge et liaison série

1.1.2 Compilation et exécution

J'ai utilisé SimulIDE 1.1.0-SR2 et Arduino IDE. Comme le chemin du compilateur n'était pas configuré dans SimulIDE, j'ai compilé les croquis dans Arduino IDE. L'erreur « Missing FQBN » a disparu après la sélection de la carte Arduino Uno.

Après la compilation, j'ai exporté le binaire avec Sketch → Export Compiled Binary. Dans SimulIDE, simulation arrêtée, j'ai utilisé Load firmware sur la carte pour charger le fichier .ino.hex sans le suffixe with_bootloader. Après une modification du code, j'ai exporté puis chargé à nouveau le binaire.

1.1.3 Suivi avec Git

J'ai enregistré les exercices au fur et à mesure dans Git, puis envoyé les commits sur GitHub. Le README décrit les dossiers et la procédure pour relancer une simulation.

1.2 Prise en main des sorties numériques

1.2.1 Premier circuit avec une LED

La sortie D2 commande la LED à travers une résistance de 100 Ω. La cathode de la LED est reliée à GND. Le programme configure D2 en sortie et alterne un état haut de 500 ms et un état bas de 500 ms. Un cycle dure donc 1 s, ce qui donne une fréquence de 1 Hz.

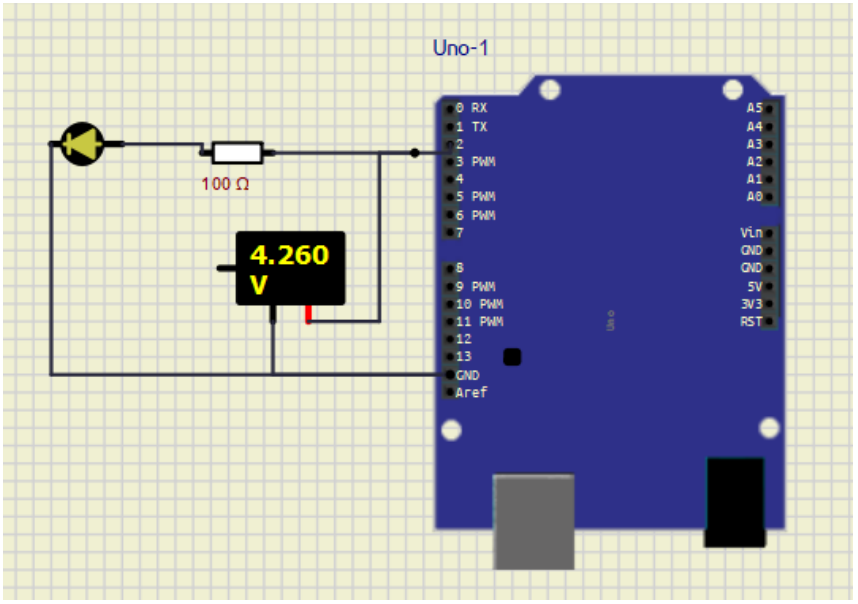


Figure 1 — LED allumée sur D2. Le voltmètre branché entre D2 et GND affiche 4,260 V à cet instant.

Lors du premier essai, le voltmètre a aidé à vérifier que D2 changeait bien d'état. La LED a clignoté après correction de son sens de branchement. La figure 1 montre un instant du cycle ; la durée du cycle découle des temporisations du programme.

1.2.2 Chenillard à dix LED

J'ai réglé le bargraphe et le réseau de résistances DIP sur dix voies. Les sorties D3 à D12 sont reliées chacune à un segment par une résistance de 100 Ω. Les cathodes sont reliées à la masse commune.

Le programme parcourt les dix sorties dans l'ordre. Il allume un segment, attend 100 ms, puis l'éteint avant d'allumer le suivant. Les segments s'allument ainsi l'un après l'autre, avec une durée nominale d'environ 1 s pour un parcours complet.

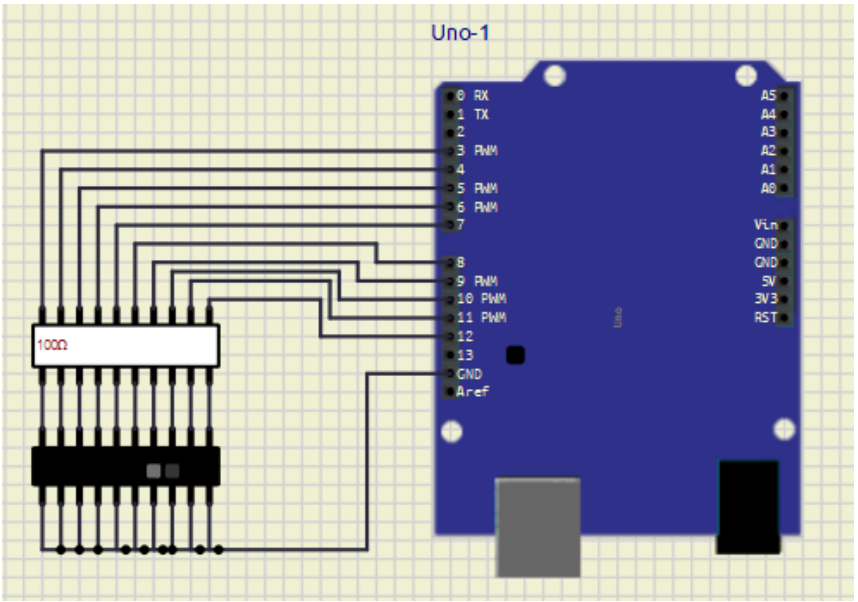


Figure 2 — Bargraphe de dix segments reliés à D3–D12 par le réseau de résistances de 100 Ω.

1.3 Lecture du DHT22 et test du seuil

Le signal DATA du DHT22 simulé est relié à A0. Le programme utilise la bibliothèque DHT sensor library d'Adafruit. La LED verte est commandée par D3 et la LED rouge par D12. Chacune est reliée en série à une résistance de 100 Ω et revient à GND.

Après la lecture du capteur, le programme compare la température à 20 °C. En dessous du seuil, seule la LED verte s'allume. À partir de 20 °C, c'est la LED rouge. En cas de lecture invalide, les deux LED sont éteintes.

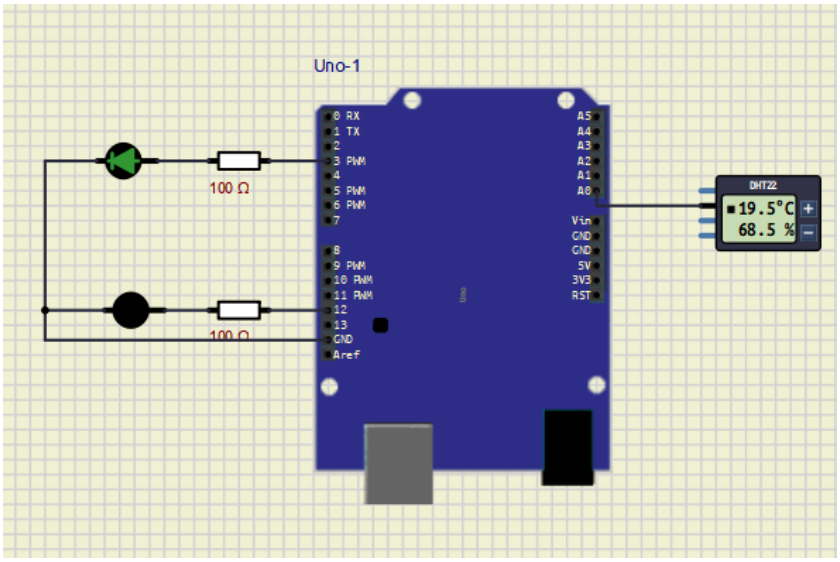


Figure 3 — À 19,5 °C, la LED verte est allumée et la LED rouge est éteinte.

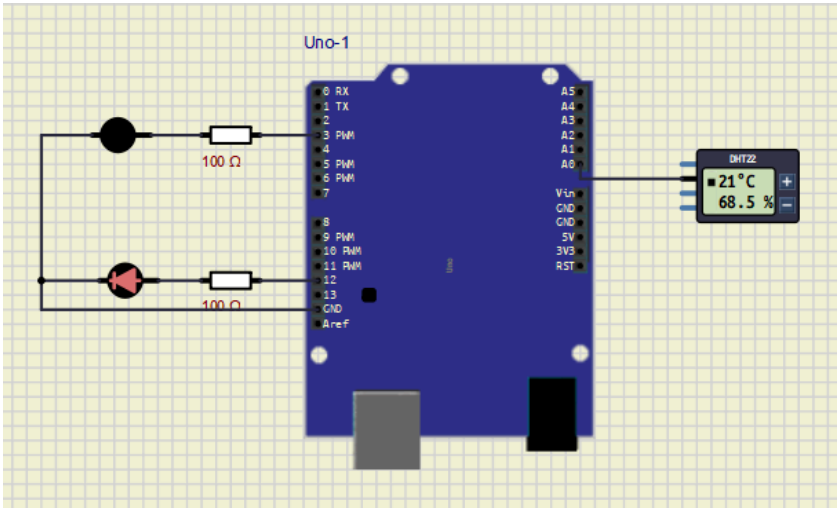


Figure 4 — À 21 °C, la LED rouge est allumée et la LED verte est éteinte.

Ces deux essais vérifient le changement d'état de part et d'autre du seuil.

1.4 Affichage des mesures sur le LCD

1.4.1 Essai du LCD seul

Le LCD Aip31068 de 16 colonnes et 2 lignes communique en I2C. J'ai relié SDA à A4 et SCL à A5, puis initialisé l'écran à l'adresse 0x3E avec la bibliothèque LiquidCrystal AIP31068 et l'en-tête LiquidCrystal_AIP31068_I2C.h. Le test place HELLO et WORLD sur deux lignes distinctes avec setCursor.

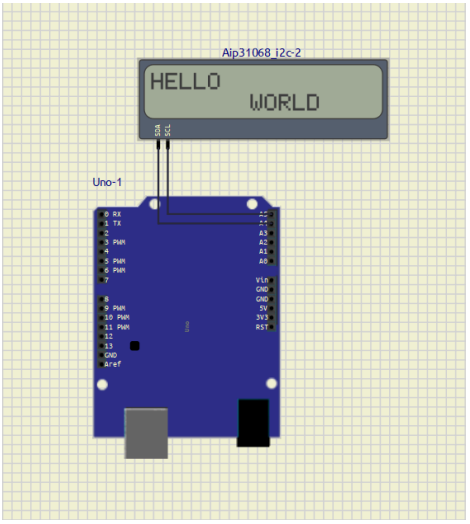


Figure 5 — HELLO apparaît sur la première ligne et WORLD sur la seconde, décalé vers la droite.

1.4.2 Association du LCD et du DHT22

Le DHT22 reste sur A0, tandis que le LCD utilise A4 et A5. La première ligne affiche « IMT NE - PPI ». La deuxième ligne reçoit les deux mesures à une décimale. Elle est effacée avant d'être réécrite pour éviter de conserver des caractères d'une valeur précédente.

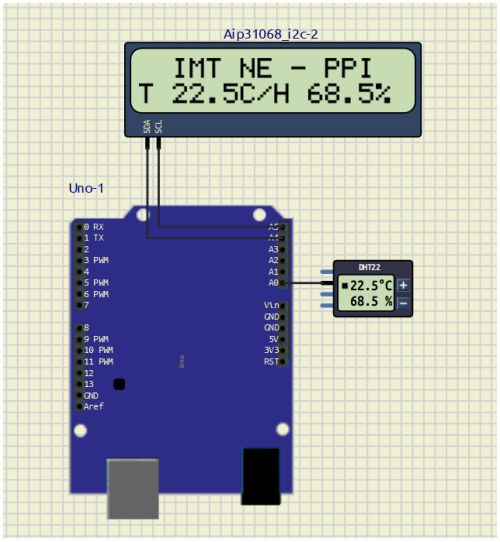


Figure 6 — Le LCD affiche 22,5 °C et 68,5 %, les mêmes valeurs que le DHT22 simulé.

Les circuits réalisés isolent trois fonctions : commander une sortie, acquérir une mesure et afficher une information. Le test du LCD seul confirme l'affichage avant d'y envoyer les mesures. Les deux composants utilisent des broches distinctes, ce qui permet de les réunir dans TempHumiX.

1.5 Montage complet de TempHumiX

Le montage rassemble le DHT22, le LCD, le bargraphe, les LED de vie, Power, TX et RX, et le terminal série. Le circuit et le programme sont enregistrés dans le dossier SIMULIDE/TempHumiX.

Élément	Raccordement dans la simulation
DHT22	DATA sur A0
LCD I2C	SDA sur A4 et SCL sur A5
LED de vie	D2 → résistance de 50 Ω → anode ; cathode à GND
Bargraphe	D3 à D12 vers les anodes via le réseau de 100 Ω ; cathodes à GND
LED Power	5 V → résistance de 50 Ω → anode ; cathode à GND

Élément	Raccordement dans la simulation
Liaison série	A1 (TX Arduino) vers RX du terminal ; TX du terminal vers A2 (RX Arduino)
LED TX et RX	Anodes reliées au 5 V via 50 Ω ; cathodes sur A1 et A2

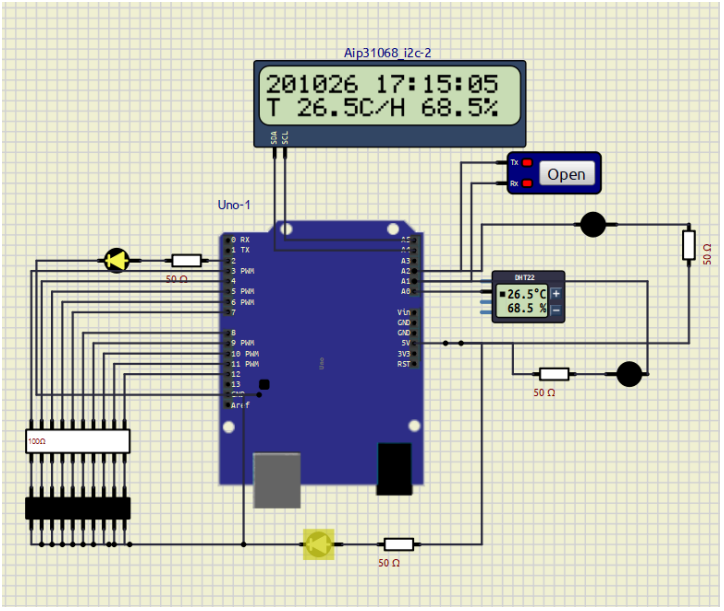


Figure 7 — Montage complet en fonctionnement. Le LCD affiche le 20/10/2026 à 17:15:05 ainsi que 26,5 °C et 68,5 %.

La LED Power est alimentée en permanence. Les indicateurs TX et RX sont câblés pour s'allumer à l'état bas sur leur ligne. Le bargraphe est présent dans le montage, mais ses sorties restent à l'état bas dans le programme d'acquisition : il n'est pas utilisé pour représenter une mesure à cette étape.

1.6 Programme d'acquisition

Au démarrage, le programme configure les sorties, initialise le capteur et le LCD, puis ouvre la liaison SoftwareSerial à 9 600 bauds. La déclaration SoftwareSerial liaison(A2, A1) indique la broche de réception, puis celle d'émission.

1.6.1 Gestion du temps dans la boucle principale

J'ai utilisé millis() pour gérer les différentes périodes. À chaque tour de boucle, le programme compare le temps écoulé au délai de chaque tâche. Il peut ainsi actualiser l'horloge et la LED de vie entre deux lectures du DHT22. Le sujet suggère un pas de 100 ms avec delay(100) ; des repères séparés permettent ici d'utiliser directement un intervalle de 250 ms pour la LED.

Action	Intervalle	Résultat
Inverser la LED de vie	250 ms	250 ms allumée + 250 ms éteinte : période de 500 ms, soit 2 Hz.
Avancer le dateur	1 000 ms	Ajout d'une seconde et actualisation de la première ligne.
Lire le DHT22	2 000 ms	Actualisation des mesures et émission d'un message série.

1.6.2 Dateur logiciel

La date et l'heure sont initialisées dans le programme. À 60 secondes, le compteur des secondes revient à zéro et les minutes avancent. Le même principe s'applique aux heures, puis aux jours. Les mois comptent 30 jours selon la simplification du sujet. Après le douzième mois, le mois revient à 1 et l'année augmente. La date revient à sa valeur initiale au redémarrage de la simulation.

1.6.3 Affichage et transmission

La première ligne du LCD utilise le format JJMMAA HH:MM:SS, qui tient sur 16 colonnes. La deuxième ligne affiche la température et l'humidité au dixième. Les valeurs sont vérifiées avant leur affichage ; une lecture invalide donne un message d'erreur. Sur le terminal, la date est envoyée avec ses séparateurs et l'année complète.

Exemple de bloc reçu dans le terminal :

```
20/10/2026 17:15:58
Temperature : 28.0 C
Humidite : 68.5 %
```

Le contrôle du passage à la minute suivante est visible dans la figure 8. Les changements de jour, de mois et d'année sont prévus par le programme ; ils ne sont pas démontrés par les captures de ce compte rendu.

1.7 Vérification de la liaison série et bilan

1.7.1 Adaptation du test COM8

Le sujet demande d'envoyer les valeurs sur COM8. Sur mon PC, aucun port COM n'était détecté et le composant Serial Port affichait « Cannot Open Port ». J'ai utilisé Serial Terminal à la place, avec les lignes RX et TX croisées. Les échanges sont ainsi observables dans SimulIDE à 9 600 bauds, 8 bits de données, sans parité et avec un bit d'arrêt. Le test valide l'émission série simulée ; le port COM8 de Windows n'a pas été testé.

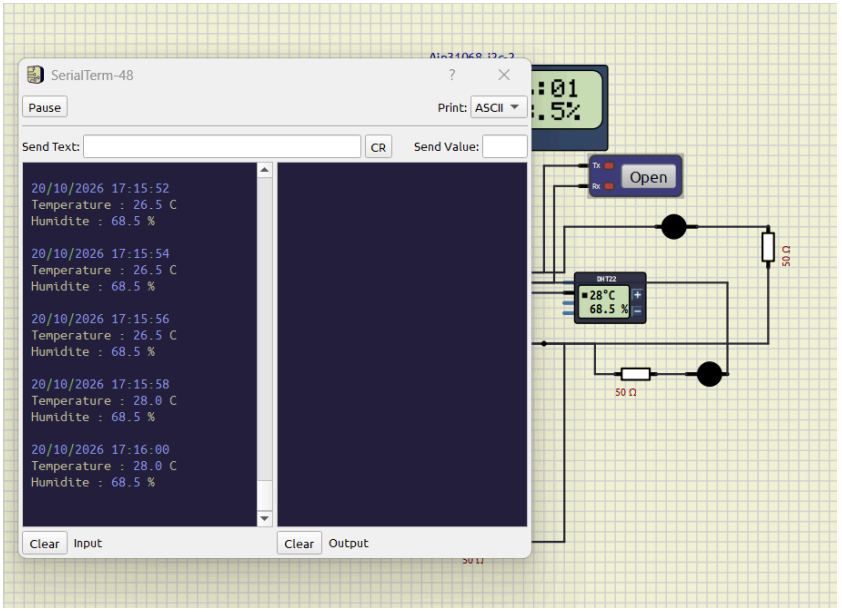


Figure 8 — Le terminal reçoit un bloc toutes les deux secondes. À 17:15:58, la température passe de 26,5 à 28,0 °C ; l'humidité reste à 68,5 %.

Les horodatages vont de 17:15:52 à 17:16:00 par pas de deux secondes. Le message de 17:16:00 confirme le passage à la minute suivante et conserve la nouvelle température. Le DHT22 visible à droite indique lui aussi 28 °C et 68,5 %.

1.7.2 Bilan de la simulation

Les essais séparés m'ont permis de vérifier les entrées et les sorties avant de construire le montage complet. La simulation affiche les mesures et les envoie au terminal avec un horodatage. Les difficultés rencontrées portaient surtout sur la polarité de la première LED, le chargement du firmware et l'ouverture du port série.

2. Communication Wi-Fi et protocole MQTT

2.1 Objectif et environnement

L'objectif du TP est de faire communiquer le boîtier TempHumiX et un PC avec le protocole MQTT. Le capteur utilise une Arduino UNO R4 WiFi, un DHT22, un afficheur LCD et un bargraphe. Les échanges sont testés avec MQTTX, puis observés dans Wireshark.

Réseau utilisé : broker Mosquitto 192.168.1.1:1883 ; adresse de l'Arduino pendant les essais : 192.168.1.101 ; PC : 192.168.1.107. Les topics de test utilisent le suffixe 48763.

2.2 Connexion et premiers échanges avec MQTTX (5.2)

La connexion au broker a été établie dans MQTTX. Une publication sur TESTS/Test_48763 a été reçue après souscription au même topic. Une publication marquée Retain a été transmise automatiquement lors d'un nouvel abonnement.

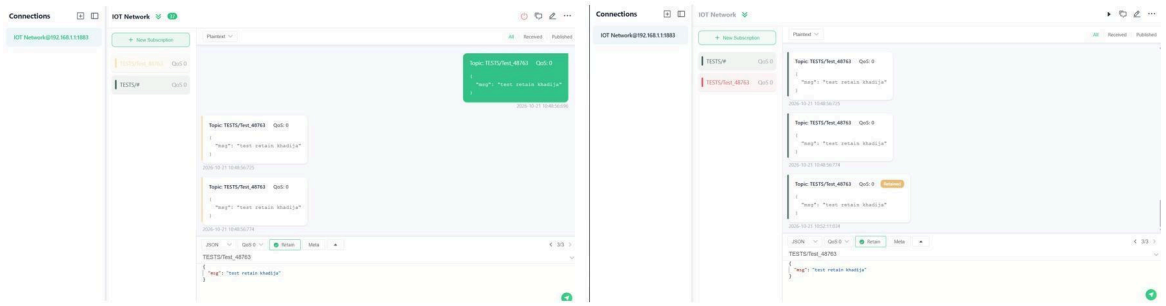


Figure 9 — Publication/souscription dans MQTTX et réception d'un message Retained.

2.2.1 Questions de la partie 5.2.4

La souscription à un topic d'un autre poste permet de recevoir ses messages si les ACL du broker l'autorisent : nous avons vu des publications d'autres postes. Le filtre TESTS/# reçoit les messages des niveaux inférieurs, alors que TESTS/+ ne correspond qu'à un niveau. Enfin, l'option Retain conserve le dernier message d'un topic et le renvoie à un nouvel abonné. Ce comportement a été observé dans MQTTX avec l'étiquette « Retained ».

2.3 Programmation de la carte (6.1.1 à 6.1.3)

2.3.1 Acquisition, LCD et RTC

Le programme initial de SimulIDE a été adapté à l'UNO R4 WiFi. Le DHT22 est lu environ toutes les deux secondes. L'écran Grove 16×2 affiche d'abord « IMT NE - PPI », puis la date, l'heure et les mesures après intégration du RTC. La LED de vie clignote à 2 Hz. Le programme garde la machine d'état et le delay(500) demandés.

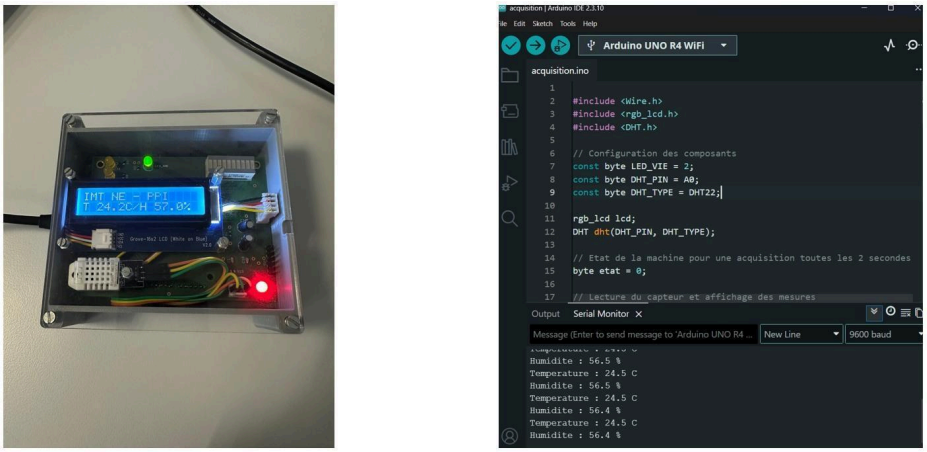


Figure 10 — Affichage LCD initial et relevés DHT22 dans le moniteur série.



Figure 11 — Date et heure fournies par le RTC, avec température et humidité.

2.3.2 Connexion Wi-Fi et synchronisation NTP

La carte a rejoint le Wi-Fi du TP et obtenu l'adresse 192.168.1.101. Le RTC est ensuite synchronisé par NTP sur 192.168.1.254, l'adresse corrigée par l'enseignant. En cas d'échec, les mesures restent affichées et une nouvelle tentative est possible.



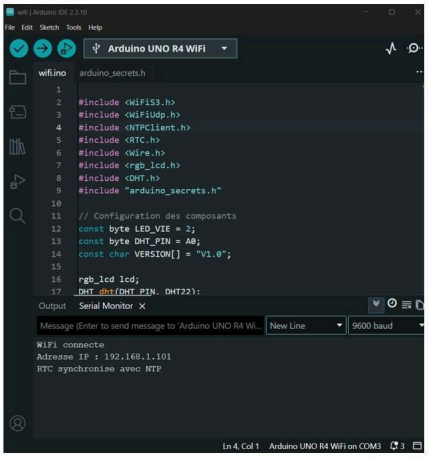


Figure 12 — Adresse IP sur le LCD et confirmation de connexion dans le moniteur série.

2.4 Publication et commandes MQTT (6.1.4 et 6.1.5)

L'Arduino publie VERSION, TEMPERATURE, HUMIDITY et GET_DATE sous les topics /iot/..._48763. Dans MQTTX, les quatre valeurs ont été reçues ; les mesures observées étaient de l'ordre de 27,3 °C et 52,8 % d'humidité.

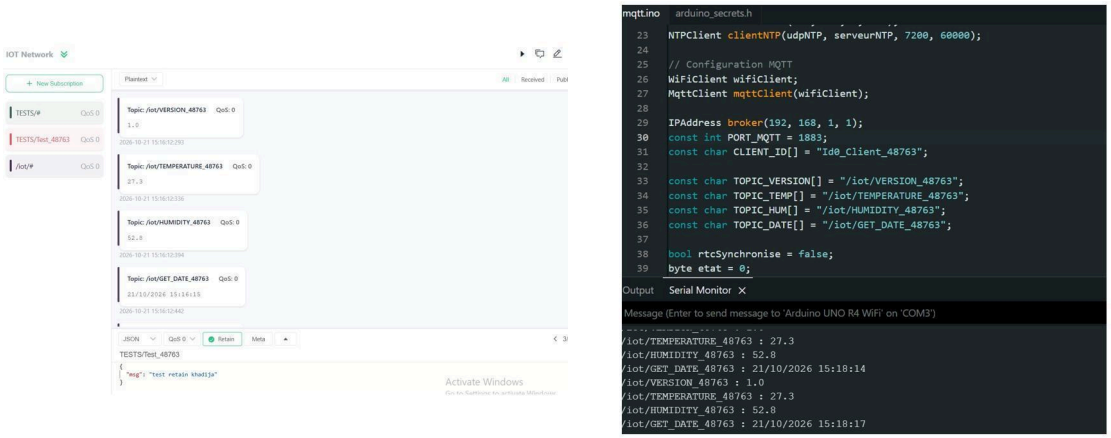


Figure 13 — Les quatre publications de l'Arduino, visibles dans MQTTX et dans le moniteur série.

L'Arduino s'abonne aux topics /iot/COMMAND_48763, /iot/BARGRAPH_48763 et /iot/BLINK_48763. La fonction onMqttMessage() récupère le topic et convertit le contenu reçu en entier. La commande 10 synchronise l'horloge, 20 démarre les mesures et 30 les arrête. Les commandes 40, 50 et 60 commandent respectivement un clignotement lent (1 000 ms), rapide (200 ms) ou personnalisé. Avec BLINK=600, la période de 600 ms a été prise en compte ; BARGRAPH=1023 a allumé les dix LED.

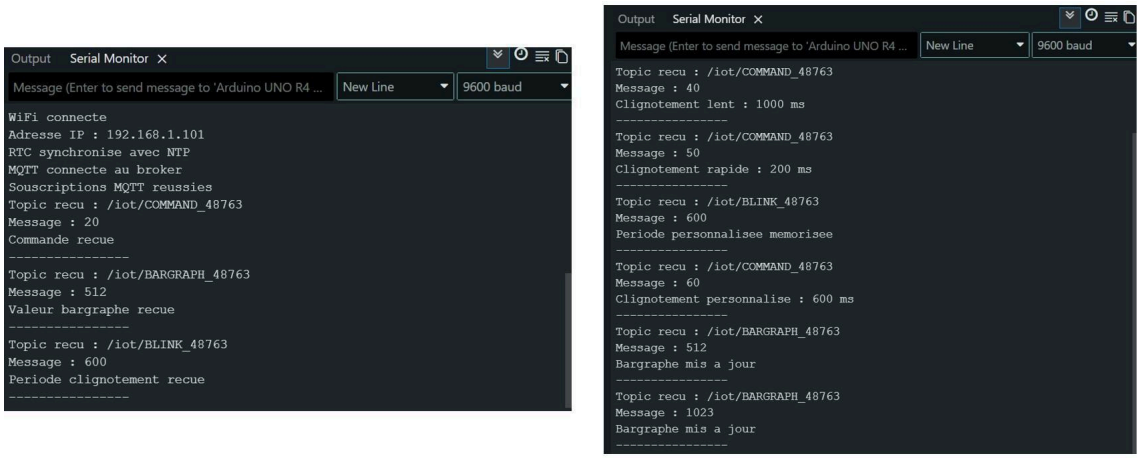


Figure 14 — Commandes envoyées et confirmations de réception dans le moniteur série.



Figure 15 — Bargraphe : les dix LED sont allumées avec la valeur 1023.

2.5 Réduction des publications (6.1.6)

Dans la première version, VERSION, TEMPERATURE, HUMIDITY et GET_DATE étaient publiées environ toutes les deux secondes, même si les valeurs étaient identiques. Nous avons ensuite modifié le programme pour publier VERSION seulement après une synchronisation NTP déclenchée par la commande 10. La température et l'humidité sont comparées aux valeurs précédentes : quand au moins une des deux change, les topics TEMPERATURE, HUMIDITY et GET_DATE sont publiés ensemble. Cela évite les envois répétitifs de valeurs inchangées et réduit le trafic MQTT. La figure montre la commande 10 et la publication de VERSION ; la condition sur les mesures est vérifiable dans le code.

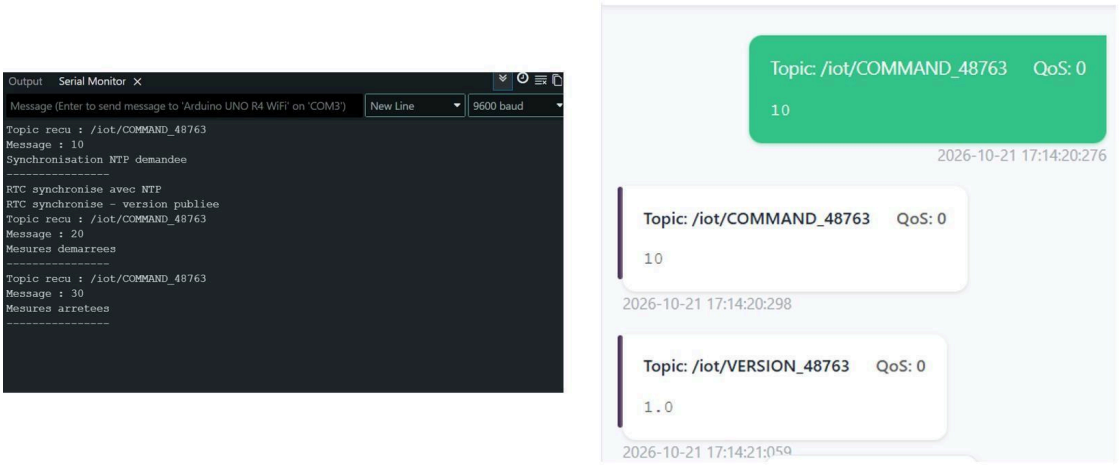


Figure 16 — Commande NTP 10, réponse série et réception de VERSION dans MQTTX.

2.6 Analyse du trafic avec Wireshark (6.2.1)

La capture a été réalisée sur l'interface Wi-Fi du PC (192.168.1.107) pendant les échanges avec le broker Mosquitto (192.168.1.1:1883). Nous avons retrouvé l'ouverture de session (CONNECT/CONNACK), les abonnements (SUBSCRIBE/SUBACK), les publications et la fermeture volontaire (DISCONNECT). Les messages PINGREQ/PINGRESP figurent aussi dans la capture précédente. Le tableau suivant relève les champs demandés, et pas seulement le rôle des paquets.

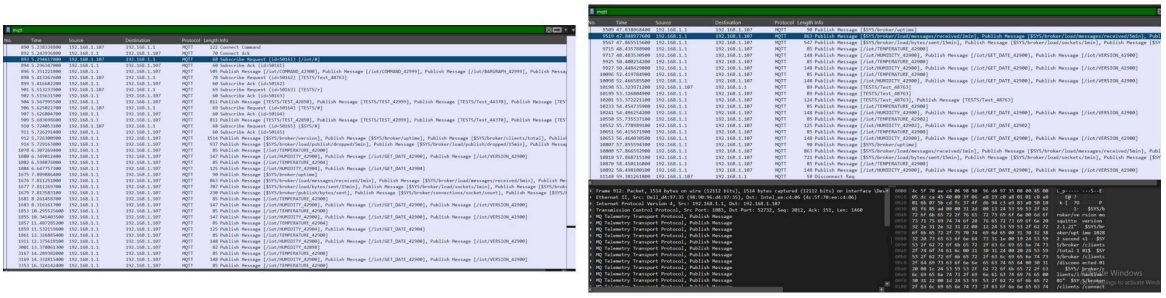


Figure 17 — Ouverture de session, abonnements et fermeture de connexion MQTT.

Paquet	Ce qu'il signifie	
CONNECT / CONNACK	Demande de connexion et réponse du broker	
SUBSCRIBE / SUBACK	Demande d'abonnement et confirmation	
PUBLISH	Envoi ou réception d'une valeur sur un topic	
PINGREQ / PINGRESP	Vérification d'une session inactive	
DISCONNECT	Fermeture volontaire de la session	
Paquet	Champ demandé	Valeur / observation relevée
CONNECT	Client ID, Keep Alive, User Name, Password	MQTT 5.0 ; Keep Alive 60 s ; authentification du TP : mos_user / mos_user. Le Client ID est présent dans CONNECT ; ne pas le déduire du topic.
CONNACK	Code de retour	Success / code 0 : connexion acceptée par le broker.
SUBSCRIBE	Topic Filter, QoS demandé	TESTS/Test_48763 et /iot/# ; QoS demandé 0 dans les essais d'abonnement.
SUBACK	QoS accordé	Abonnements confirmés (SUBACK) ; QoS 0 pour les filtres montrés.
PUBLISH	Topic, Message, QoS, Retain, DUP	Ex. /iot/TEMPERATURE_48763, « 26.5 », QoS 0 ; Retain et DUP sont des

Paquet	Ce qu'il signifie	
		indicateurs du PUBLISH. Pour TESTS/Test_48763, QoS 1 et QoS 2 ont été observés séparément.
PINGREQ/PINGRESP	Échanges et périodicité	Présents dans la capture ; Keep Alive annoncé : 60 s. L'intervalle réel dépend de l'activité MQTT.
DISCONNECT	Fin de session	Client (192.168.1.107) → broker (192.168.1.1) ; fermeture volontaire observée (trame 11148).

2.6.1 Réponses aux questions Wireshark

Mot de passe : dans la configuration du TP, le client se connecte avec le nom d'utilisateur « mos_user » et le mot de passe « mos_user ». Ces informations sont transportées par CONNECT. Comme nous utilisons MQTT sur TCP 1883 sans TLS, elles peuvent être lues par une personne capable d'intercepter le trafic. Pour un usage réel, il faudrait utiliser TLS, des identifiants individuels et des ACL restrictives.

Taille : un PUBLISH de température sur /iot/TEMPERATURE_48763 occupe 31 octets au niveau MQTT et 85 octets pour la trame Ethernet complète (exemple de la capture, valeur 26.5). Il faut bien distinguer charge MQTT et trame réseau. Une requête HTTP équivalente contient une ligne de requête et des en-têtes ; elle est généralement plus volumineuse, mais aucune requête HTTP de référence n'a été mesurée ici. Nous n'en donnons donc pas de taille prétendument observée.

Keep Alive : la connexion MQTTX relevée annonce 60 secondes. Quand aucun autre paquet MQTT n'est envoyé, le client peut transmettre PINGREQ et recevoir PINGRESP pour confirmer que la liaison fonctionne. Dans la capture, ces messages sont présents (ex. trames 14028 et 14029). Si aucun paquet ne parvient au broker pendant environ 1,5 fois le Keep Alive, soit 90 secondes ici, il peut fermer la connexion.

2.7 QoS, Retain et abonnements génériques (6.2.2)

Les niveaux QoS ont été comparés en publiant le même type de message sur TESTS/Test_48763, puis en observant les trames. Au QoS 0, il n'y a pas d'acquittement MQTT ; au QoS 1, PUBLISH est suivi de PUBACK ; au QoS 2, l'échange comprend PUBLISH, PUBREC, PUBREL et PUBCOMP.

QoS	Trames MQTT	Livraison
0	PUBLISH	Au plus une fois
1	PUBLISH + PUBACK	Au moins une fois (doublons possibles)
2	PUBLISH + PUBREC + PUBREL + PUBCOMP	Exactement une fois

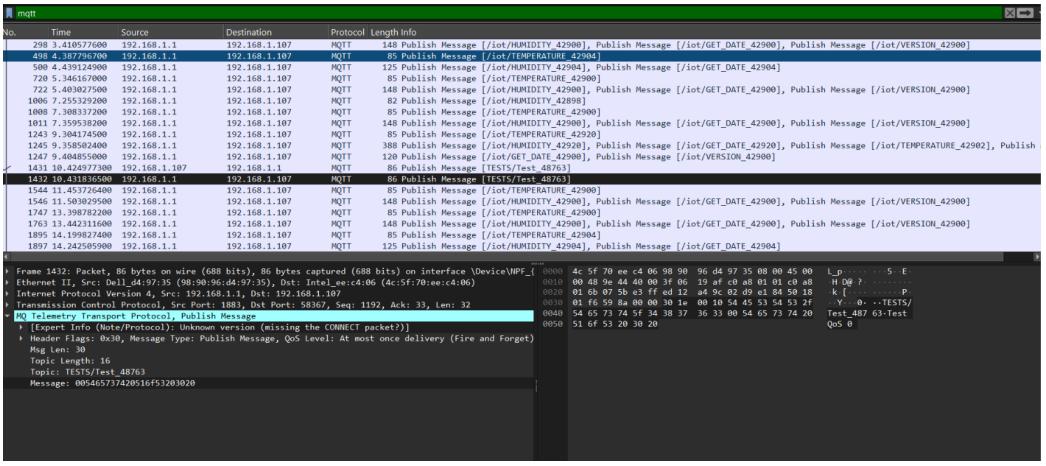


Figure 18 — Test QoS 0 : paquet PUBLISH sans accusé de réception MQTT.

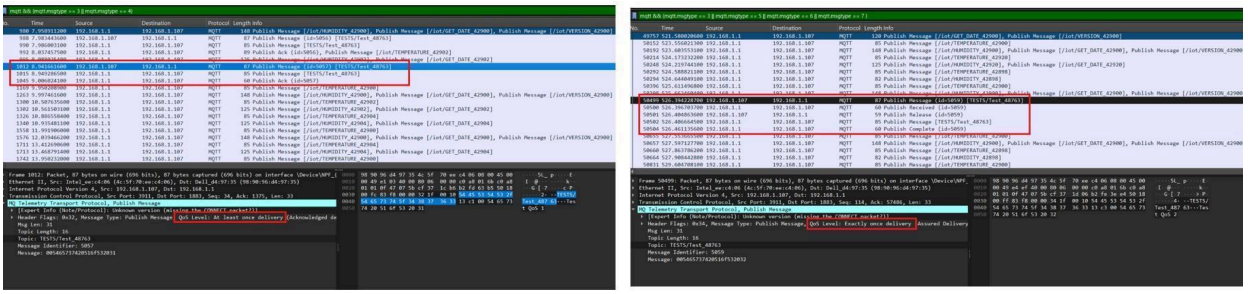


Figure 19 — QoS 1 (PUBLISH/PUBACK, identifiant 5057) et QoS 2 (quatre étapes, identifiant 5059).

Pour Retain, une publication conservée a bien été reçue après abonnement. Une publication de charge utile vide avec Retain activé a ensuite effacé cette valeur : à la souscription suivante, aucun ancien message Retained n'a été reçu.

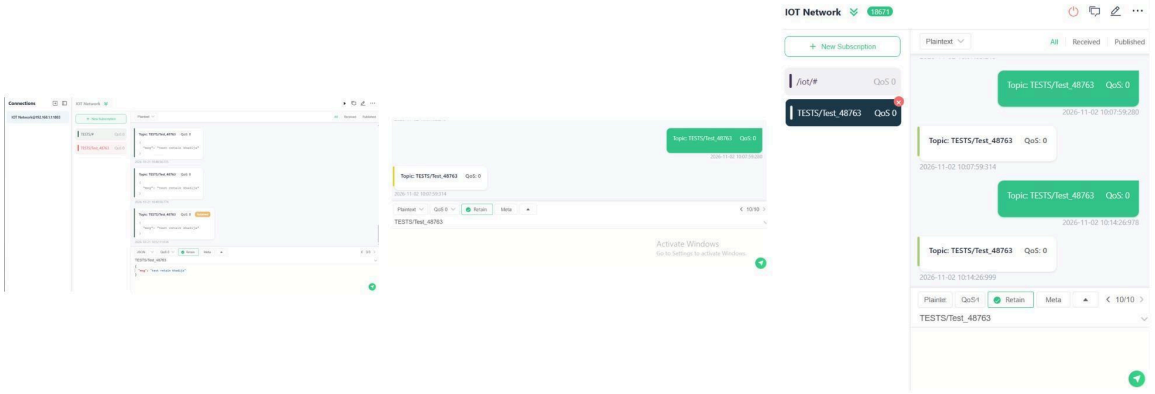


Figure 20 — Message retenu, publication vide avec Retain et contrôle après réabonnement.

Les filtres TESTS/+ et TESTS/# ont été testés. Le premier correspond à un seul niveau sous TESTS ; le second accepte aussi des niveaux imbriqués, comme TESTS/Capteur/Temperature. L'abonnement $SYS/# fournit des données du broker (compteurs de messages, octets, connexions). Plusieurs abonnements étaient actifs simultanément : les captures montrent les messages reçus, tandis que la différence entre + et # découle de la règle de filtrage MQTT.

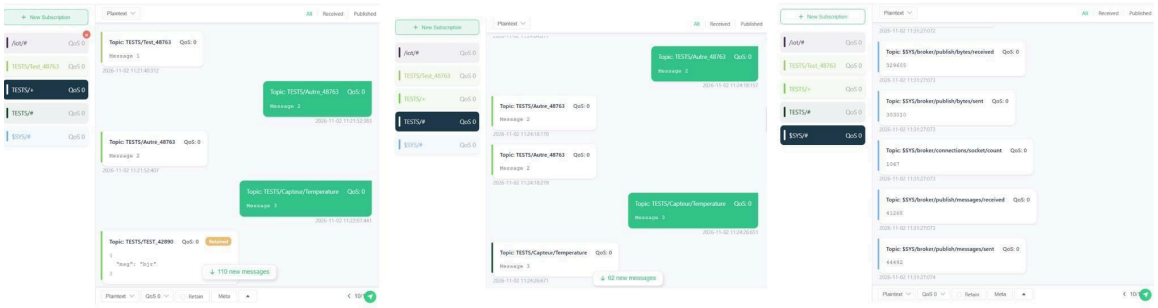


Figure 21 — Abonnements TESTS/+, TESTS/# et statistiques $SYS/#.

2.8 Questions générales — pages 15 et 16

2.8.1 Acheminement et modèle OSI / TCP-IP

Le DHT22 fournit la température et l'humidité à l'Arduino. Celui-ci construit un message MQTT comprenant un topic et une valeur, puis l'envoie par TCP/IP à travers le Wi-Fi. Le broker Mosquitto reçoit le message et le redistribue aux clients abonnés (ici MQTTX). Dans le modèle OSI, MQTT appartient à la couche application (7), TCP au transport (4), IP au réseau (3), et Wi-Fi ou Ethernet aux couches liaison/physique (2/1). Au niveau TCP/IP, cela correspond aux couches application, transport, Internet et accès réseau. Le RUTX50 fournit le point d'accès et les fonctions réseau nécessaires. Pour des appareils sur un même LAN, il n'est pas indispensable de passer par Internet ; le routage n'intervient que si les réseaux sont distincts.

2.8.2 Services du routeur et place dans l'IoT

Le RUTX50 propose des services et fonctions de plusieurs couches. Parmi les services applicatifs, on trouve notamment DHCP, DNS/DNS relay, une interface Web d'administration, des fonctions de supervision (SNMP) et, selon les modules installés, des clients/protocoles IoT comme MQTT. Le NAT, le pare-feu, les VLAN et le routage relèvent plutôt des fonctions réseau, et ne doivent pas être classés comme des services applicatifs. Dans notre montage, le routeur est un équipement de proximité (edge/fog pour l'accès et les traitements locaux), et non un cloud : il relie les objets et les postes, éventuellement à Internet ou à un cloud extérieur. Il assure le rôle de point d'accès Wi-Fi, de routeur/passerelle entre sous-réseaux et éventuellement de pont selon la configuration. Mosquitto est un service applicatif distinct, même s'il est hébergé sur la même infrastructure.

2.8.3 Microcontrôleur ou microprocesseur ?

Le RUTX50 repose sur un microprocesseur applicatif (MPU), et non sur un simple microcontrôleur. Le constructeur indique un processeur ARM Cortex-A7 quadricœur à 717 MHz, avec 256 Mo de RAM et RutOS, un système Linux basé sur OpenWrt. Il peut donc exécuter plusieurs services réseau simultanément. À l'inverse, l'Arduino UNO R4 exécute principalement notre programme embarqué sur microcontrôleur.

2.8.4 Nombre de connexions Wi-Fi, risques et topologie

Les connexions Wi-Fi ne sont pas illimitées : le nombre d'équipements admis dépend des ressources du point d'accès, de la bande passante, de la qualité radio et des paramètres de configuration. Plus les clients sont nombreux, plus les risques de saturation, collisions radio et baisse de débit augmentent. Côté sécurité, un mot de passe partagé entre de nombreux postes peut être diffusé, et des clients mal protégés peuvent donner accès au réseau. La topologie est une topologie Wi-Fi en infrastructure (étoile logique autour du point d'accès du RUTX50), avec un broker sur le réseau local. Il n'est pas possible de déduire une capacité réelle de connexions simultanées de nos seules captures.

2.8.5 Sécurité du RUTX50

Les protections disponibles sur le RUTX50 comprennent l'authentification Wi-Fi (WPA2/WPA3 selon la configuration), le pare-feu, la segmentation VLAN, les VPN (par exemple WireGuard, OpenVPN et IPsec), les contrôles d'accès et la protection contre certaines attaques réseau. La documentation du constructeur cite également les certificats et l'administration sécurisée. Au 3 novembre 2026, Teltonika indique la version RUTX_R_00.07.26.1 (29 octobre 2026) comme dernier firmware publié pour ce modèle ; son journal mentionne notamment un correctif de vulnérabilité CVE-2026-114224. Cela ne signifie pas que le routeur du TP utilisait cette version : nous n'avons pas vérifié son firmware ni les options activées. La communication MQTT de nos essais reste non chiffrée sur 1883.

2.8.6 Analyse des logs Mosquitto

Les logs indiquent le démarrage de Mosquitto 2.1.2 et le chargement de /mosquitto/config/mosquitto.conf. Le broker signale la disponibilité du support bridge, persistence, TLS, TLS-PSK et WebSockets ; cela ne prouve pas que tous ces services sont activés. Le processus s'exécute avec l'utilisateur système « mosquitto ». À ce démarrage, zéro message de base, zéro message retenu, zéro client et zéro abonnement ont été restaurés.

Les avertissements signalent des permissions trop larges sur les fichiers de mots de passe et d'ACL, ainsi qu'un propriétaire incorrect pour acl. Les lignes du journal recommandent chmod 0700 sur ces fichiers et chown mosquitto pour acl : l'objectif est de limiter leur lecture et de les faire posséder par le compte du service. Le motif d'ACL « TESTS/# » ne contient pas les variables %c (identifiant client) ni %u (nom d'utilisateur) : il s'agit donc d'une règle commune plutôt que spécifique à chaque client. Enfin, le broker ouvre des sockets IPv4 et IPv6 sur le port 1883, puis annonce qu'il est en fonctionnement. Ces avertissements méritent correction même si le service démarre.

2.9 Bilan du TP MQTT

Ce TP m'a permis de passer d'une simulation à une communication réelle entre l'Arduino UNO R4 WiFi, le broker Mosquitto et MQTTX. J'ai pu publier les mesures du DHT22, récupérer l'heure par NTP et

commander la LED ainsi que le bargraphe à distance. La partie Wireshark m’a aidée à voir concrètement la différence entre QoS 0, 1 et 2 et à comprendre Retain et Keep Alive. J’ai rencontré plusieurs difficultés : adaptation de l’afficheur à la nouvelle carte, bibliothèque NTP manquante, adresse IP d’abord affichée à 0.0.0.0 et échec ponctuel d’une synchronisation. Elles ont été résolues en testant séparément les fonctions et en observant le moniteur série. La suite du projet sera la construction d’une IHM d’affichage.

3. Interface de supervision sous PySide6

3.1 Objectif et environnement

Cette dernière phase prolonge la programmation du boîtier TempHumiX, basé sur une Arduino UNO R4 WiFi et un capteur DHT22. La première partie permettait déjà de mesurer la température et l'humidité, de les afficher sur un écran LCD et de les transmettre par MQTT.

L'objectif de cette dernière étape était de créer une application sur PC capable de recevoir les mesures, d'envoyer des commandes au boîtier, d'archiver les valeurs et de consulter leur évolution sous forme de courbes.

3.1.1 Matériel et outils

Boîtier Arduino UNO R4 WiFi, DHT22, LCD et bargraphe ; broker Mosquitto sur le réseau du TP (192.168.1.1, port 1883). Développement sous Windows avec VS Code, Python, paho-mqtt, PySide6/Qt Designer, SQLite et Matplotlib.

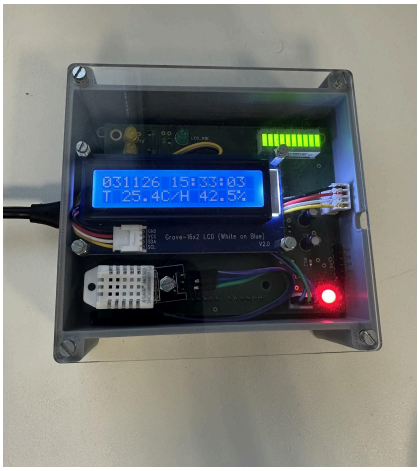


Figure 22 — Boîtier TempHumiX utilisé pendant les essais.

Les échanges utilisent les topics /iot/TEMPERATURE_48763, /iot/HUMIDITY_48763, /iot/GET_DATE_48763 et /iot/VERSION_48763. Les commandes passent par /iot/COMMAND_48763, /iot/BLINK_48763 et /iot/BARGRAPH_48763.

3.2 Communication MQTT en mode console

3.2.1 Client MQTT en lecture (sections 3.2 et 3.3)

J'ai créé la classe ClientMQTT avec la bibliothèque paho-mqtt. La méthode connect_mqtt établit la connexion au broker, disconnect_mqtt la ferme, subscribe_mqtt s'abonne aux topics et publish_mqtt transmet les commandes. La méthode wait_mqtt traite les messages entrants. Dans le premier programme console, la réception affiche la température, l'humidité et la date du capteur.

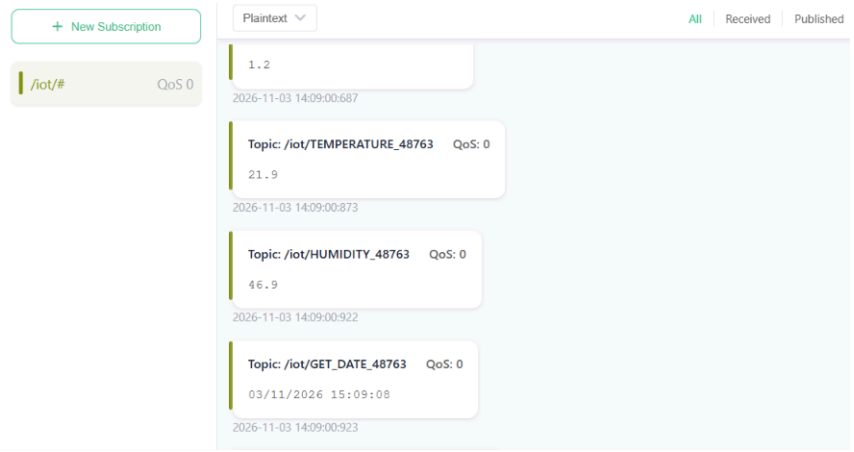


Figure 23 — Contrôle préalable des publications du capteur avec MQTTX.

```
(.venv) PS C:\Users\ZBOOK\IOT_OK\IHM> python console_mqtt_lecture/main.py
Démarrage du client MQTT en lecture
Attente du topic : '/iot/VERSION_48763'
Attente du topic : '/iot/TEMPERATURE_48763'
Attente du topic : '/iot/HUMIDITY_48763'
Attente du topic : '/iot/GET_DATE_48763'
Attente des données (Ctrl+C pour arrêter)...
Connecté au broker MQTT : 192.168.1.1:1883
Réception : 22.2 depuis '/iot/TEMPERATURE_48763'
Réception : 46.9 depuis '/iot/HUMIDITY_48763'
Réception : 03/11/2026 15:10:22 depuis '/iot/GET_DATE_48763'
Réception : 22.2 depuis '/iot/TEMPERATURE_48763'
Réception : 47.0 depuis '/iot/HUMIDITY_48763'
Réception : 03/11/2026 15:10:35 depuis '/iot/GET_DATE_48763'
Réception : 22.2 depuis '/iot/TEMPERATURE_48763'
Réception : 46.9 depuis '/iot/HUMIDITY_48763'
Réception : 03/11/2026 15:10:37 depuis '/iot/GET_DATE_48763'
Réception : 22.2 depuis '/iot/TEMPERATURE_48763'
Réception : 46.8 depuis '/iot/HUMIDITY_48763'
Réception : 03/11/2026 15:10:42 depuis '/iot/GET_DATE_48763'
Réception : 22.2 depuis '/iot/TEMPERATURE_48763'
Réception : 46.7 depuis '/iot/HUMIDITY_48763'
```

Figure 24 — Réception des mesures dans le programme Python en lecture.

La console indique une connexion réussie à 192.168.1.1:1883. Les valeurs de température, d'humidité et de date arrivent correctement. Cela valide le fonctionnement du client en lecture. Les noms de topics ont été conservés en majuscules pour correspondre au programme Arduino utilisé.

3.3 Envoi des commandes au capteur

3.3.1 Client MQTT en écriture (section 3.4)

Un deuxième programme propose un menu de sept actions : mise à l'heure, démarrage et arrêt des mesures, clignotement lent, rapide ou personnalisé, et écriture sur le bargraphe. Chaque choix publie une valeur sur le topic MQTT correspondant.

```
(.venv) PS C:\Users\ZBOOK\IOT_OK\IHM> python console_mqtt_ecriture/main.py
Démarrage du client MQTT en écriture

Console capteur IdO
1 - Mise à l'heure du capteur
2 - Démarrage des mesures
3 - Arrêt des mesures
4 - Clignotement lent
5 - Clignotement rapide
6 - Clignotement personnalisé
7 - Écriture du bargraphe
Q - Quitter
Votre choix : Connecté au broker MQTT : 192.168.1.1:1883
4
Publication : 40 sur '/iot/COMMAND_48763'
```

Figure 25 — Envoi de la commande 40 pour le clignotement lent.



Figure 26 — Arrêt des mesures : commandes 30 puis 40.

La commande 40 a été vérifiée sur le boîtier : la LED a ralenti. L'arrêt des mesures est suivi de la commande 40 (clignotement lent), et le redémarrage de la commande 50 (clignotement rapide), conformément au sujet. Le contrôle du bargraphe a aussi été testé avec plusieurs valeurs, notamment 0 et 1023.

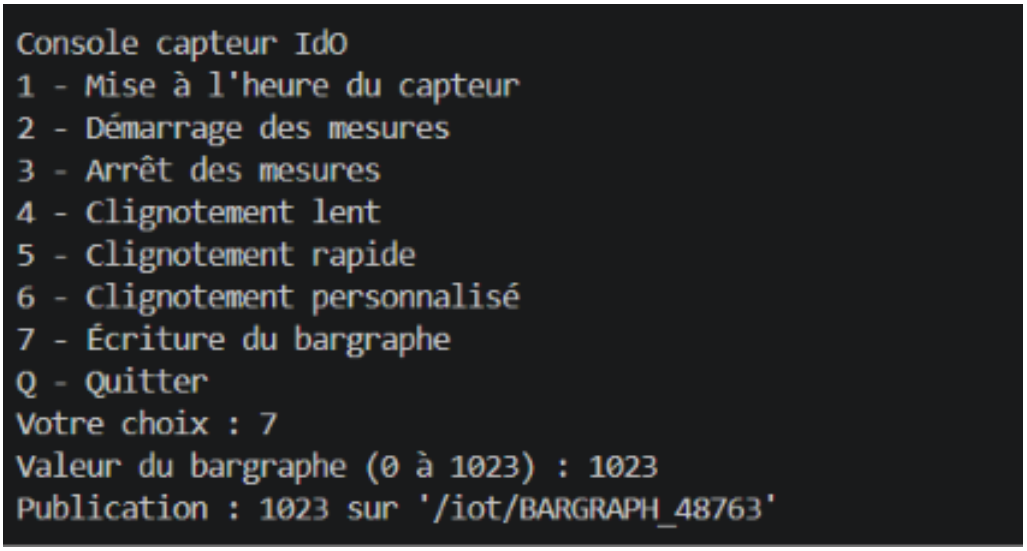


Figure 27 — Publication de 1023 sur le topic BARGRAPH.

3.4 Exécution parallèle et thread MQTT

3.4.1 Deux clients indépendants (section 3.5)

J'ai lancé le client de lecture et celui d'écriture dans deux terminaux, chacun avec un identifiant MQTT distinct (Client_48763 et Client_E48763). Deux clients ne doivent pas partager le même identifiant sur le broker. En envoyant la commande 30, puis la commande 20, j'ai pu observer l'arrêt puis la reprise des nouvelles mesures dans le terminal de lecture.

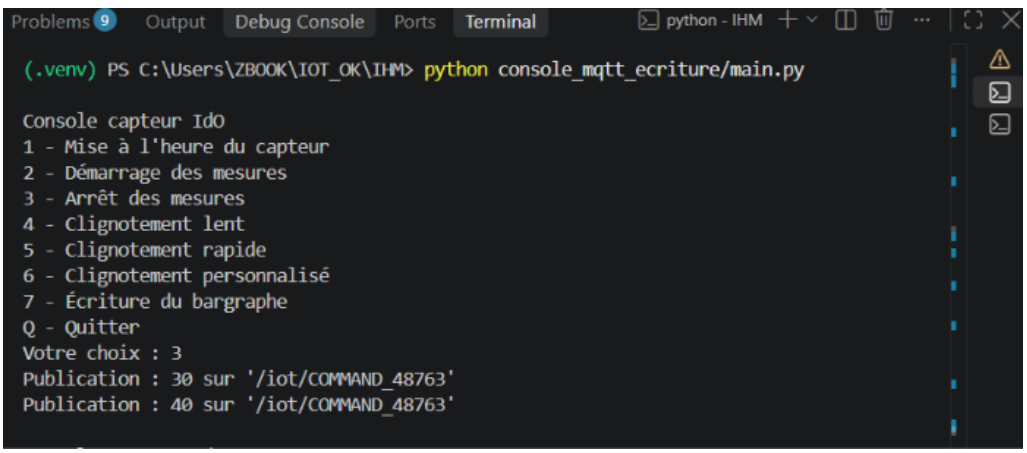


Figure 28 — Terminal d'écriture : demande d'arrêt des mesures.

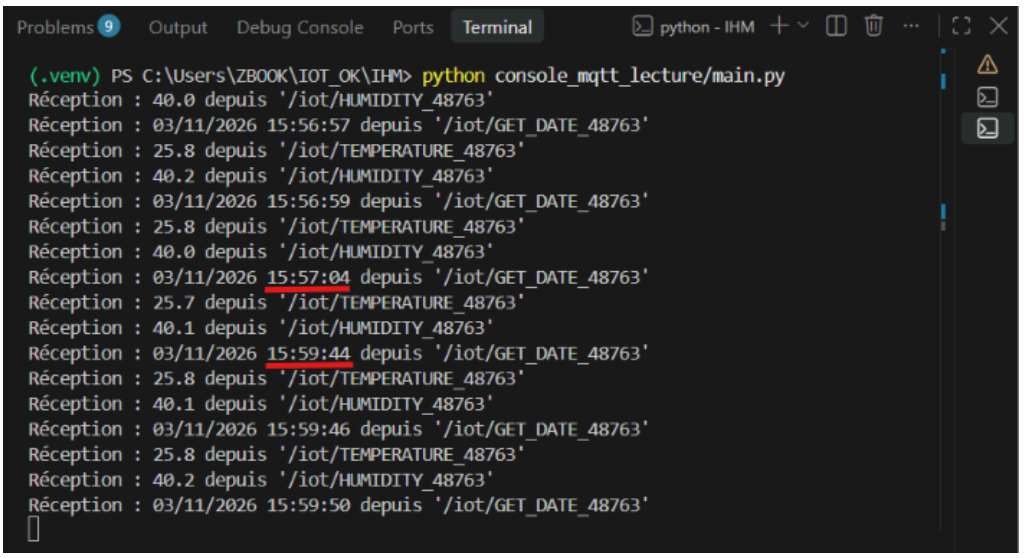


Figure 29 — Terminal de lecture : observation des messages pendant les essais.

3.4.2 Réunion dans un même programme (section 3.6)

J'ai ensuite regroupé lecture et écriture dans une seule application. La classe ThreadMQTT exécute la boucle de réception dans un thread séparé : le programme principal peut donc rester disponible pour le menu des commandes. Les dernières valeurs reçues sont conservées dans les attributs de ClientMQTT. À la fermeture, stopthread() demande l'arrêt de la boucle avant la déconnexion. Le test a confirmé que les mesures continuaient d'arriver pendant l'envoi d'une commande.

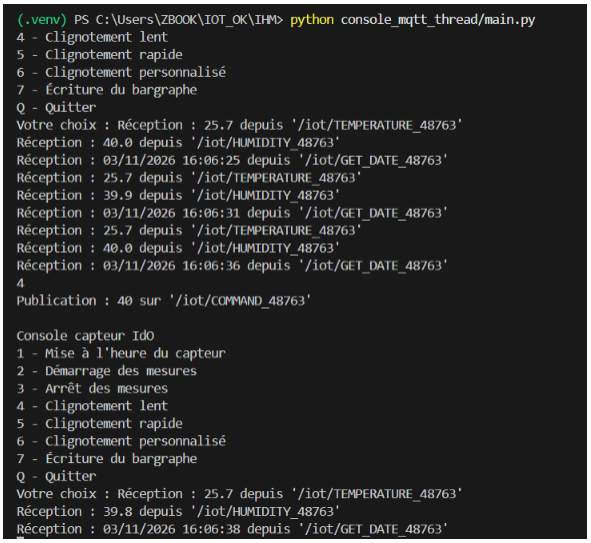


Figure 30 — Lecture MQTT et menu de commandes dans la même application.

3.5 Interface graphique sous PySide6

3.5.1 Conception et conversion (sections 4.3 et 4.4)

L'interface principale a été dessinée dans Qt Designer : champs de connexion au broker, affichage de la date, de la température et de l'humidité, boutons de commande et saisies pour le clignotement personnalisé et le bargraphe. Le formulaire mon_ihm.ui a ensuite été converti en mon_ihm.py avec pyside6-uic.

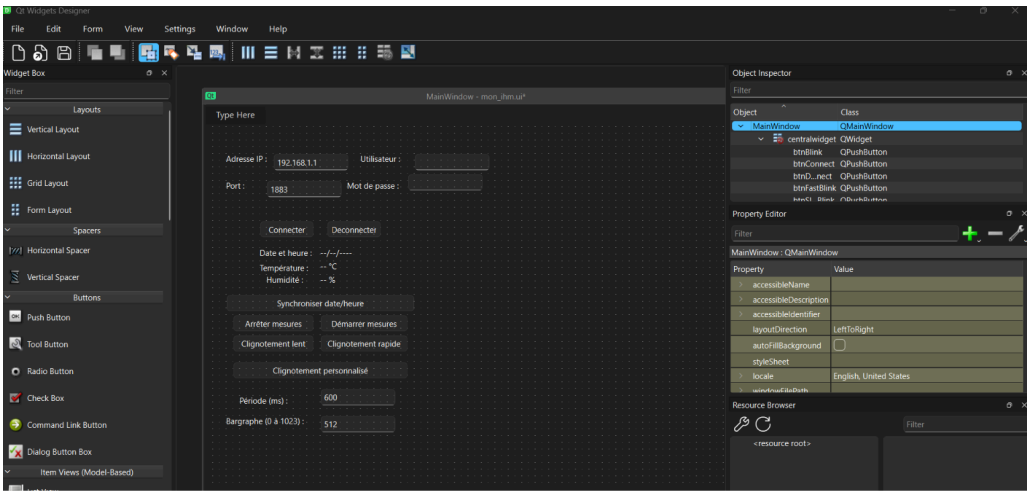


Figure 31 — Création de la fenêtre principale dans Qt Designer.

3.5.2 Fonctionnement de l'application (section 4.5)

Dans main.py, les signaux des boutons sont reliés aux méthodes Python (connexion, déconnexion et commandes MQTT). La réception s'exécute dans ThreadMQTT, tandis qu'un QTimer rafraîchit l'affichage toutes les 1 000 ms à partir des dernières valeurs mémorisées. Ainsi, la fenêtre reste utilisable même lorsque le capteur n'envoie pas de nouvelle mesure.

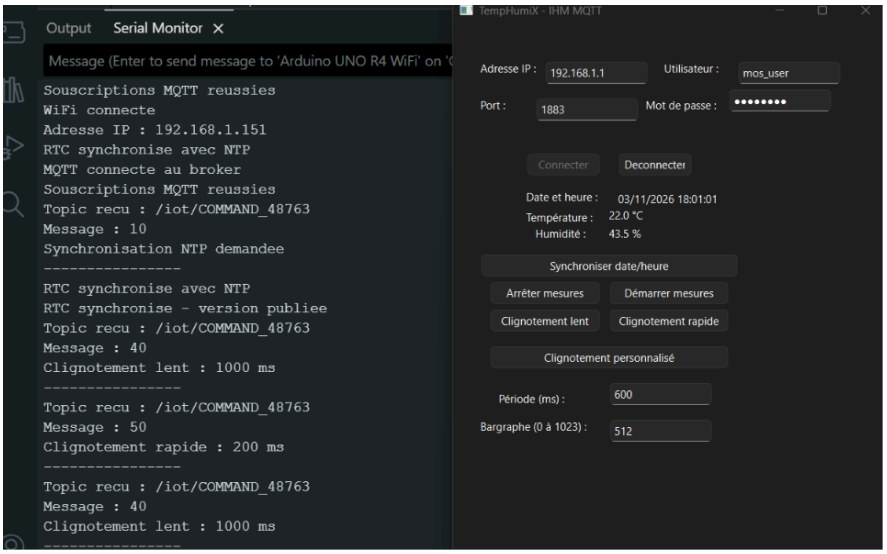


Figure 32 — Application PySide6 et moniteur série Arduino pendant les tests.

Les essais ont confirmé l'affichage des mesures réelles, la synchronisation NTP, les différents clignotements et l'écriture sur le bargraphe. Le moniteur série Arduino permet de vérifier la réception des commandes envoyées depuis l'IHM.

3.6 Notice de fonctionnement de l'IHM

Pour utiliser TempHumiX, il faut d'abord brancher le boîtier et vérifier son accès au broker MQTT du réseau IoT. L'application est lancée depuis le dossier ihm_mqtt_simple avec la commande « python main.py ».

Action dans l'IHM	Effet attendu
Renseigner IP, port et identifiants, puis « Connecter »	Ouverture de la session MQTT et affichage des dernières mesures reçues
« Synchroniser date/heure »	Demande de synchronisation NTP du boîtier (commande 10)
« Démarrer mesures » / « Arrêter mesures »	Commandes 20/50 ou 30/40 suivant l'action
Boutons « Clignotement lent / rapide »	Commandes 40 ou 50
Saisir une période et cliquer « Clignotement personnalisé »	Publication sur BLINK puis commande 60

Action dans l'IHM	Effet attendu
Saisir une valeur de bargraphe et appuyer sur Entrée	Publication d'une valeur entre 0 et 1023
« Afficher les courbes »	Ouverture de la fenêtre de consultation de l'historique
Choisir une période, puis « Charger les données »	Tracé de la température et de l'humidité sur cette période
« Effacer les courbes »	Suppression du tracé affiché

En fin d'utilisation, le bouton « Déconnecter » arrête la réception et ferme la communication.

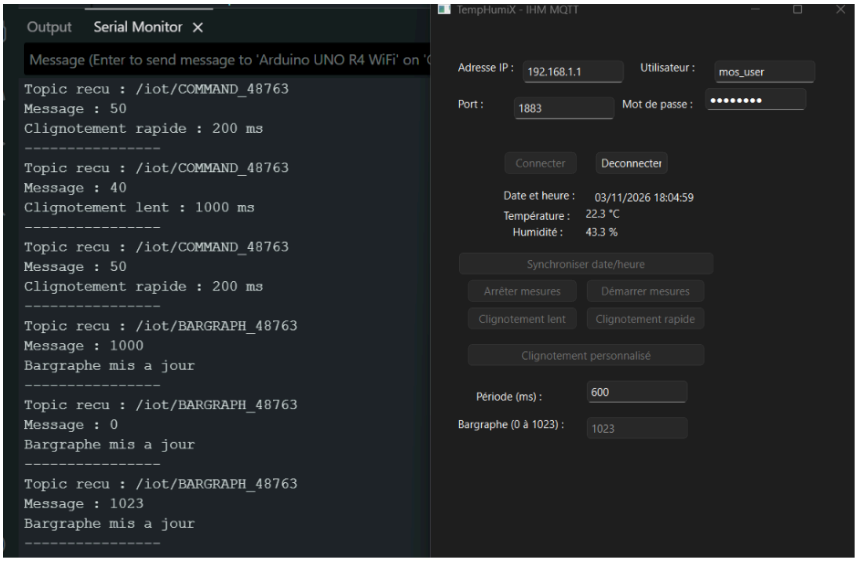


Figure 33 — Vérification des commandes et du bargraphe avec le moniteur série.

3.7 Archivage des mesures dans SQLite

Pour l'archivage (section 5 du TP IHM), j'ai ajouté l'archivage avec sqlite3. La méthode connect_sqlite() ouvre ou crée IOT.db et initialise la table IOT_DATA (Id_capteur, Date, Version, Temperature, Humidite). À chaque mesure complète, une requête INSERT ajoute une ligne et commit() valide l'enregistrement. La connexion SQLite est fermée à l'arrêt du thread avec disconnect_sqlite().

Dans mon programme Arduino, le topic GET_DATE arrive après TEMPERATURE et HUMIDITY. J'ai donc choisi de sauvegarder la ligne à la réception de GET_DATE, lorsque la date et les deux mesures sont connues. Les lignes ne sont pas forcément espacées de deux secondes, car la version optimisée de l'Arduino publie seulement lorsque les mesures évoluent.

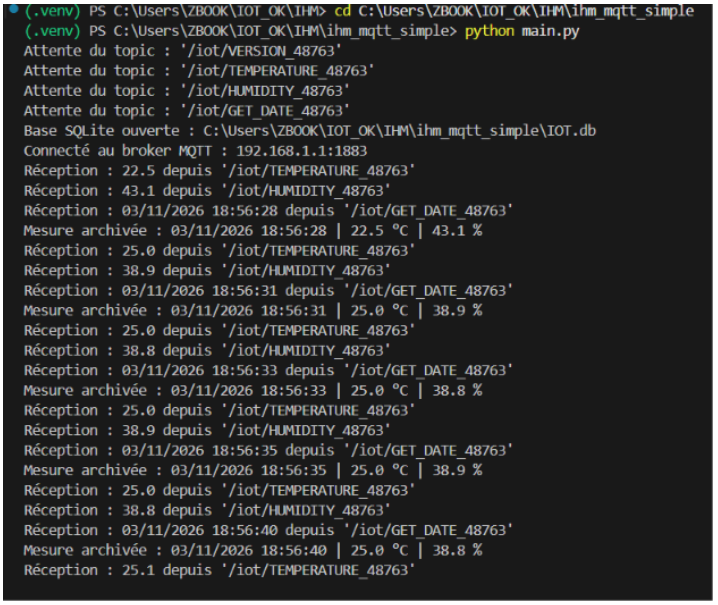


Figure 34 — Messages « Mesure archivée » affichés dans le terminal.

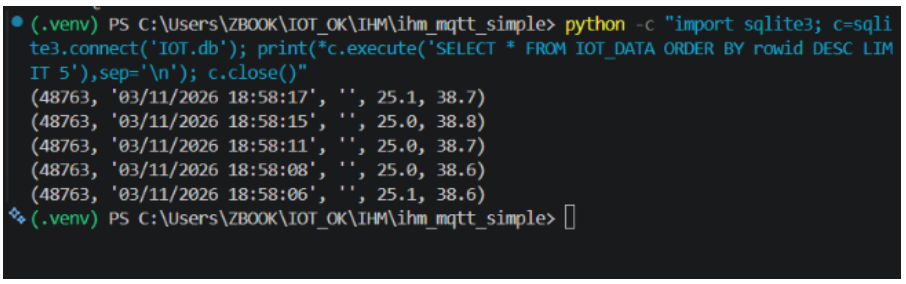


Figure 35 — Lecture SQL des cinq derniers enregistrements dans IOT_DATA.

J'ai vérifié le résultat avec une requête SQL affichant les cinq dernières lignes. Chaque enregistrement comporte bien l'identifiant du capteur, l'horodatage et les valeurs numériques de température et d'humidité. Ces données sont ensuite réutilisées pour tracer les courbes.

3.8 Consultation de l'historique sous forme de courbes

Pour l'affichage des courbes (section 6 du TP IHM), j'ai créé graphe.ui avec deux champs de date et heure, une zone graphique et les boutons « Charger les données » et « Effacer les courbes ». Dans dialogue_courbe.py, je lis les enregistrements SQLite, je convertis leurs dates au format JJ/MM/AAAA HH:MM:SS pour filtrer la période choisie, puis je trace les mesures retenues avec Matplotlib.

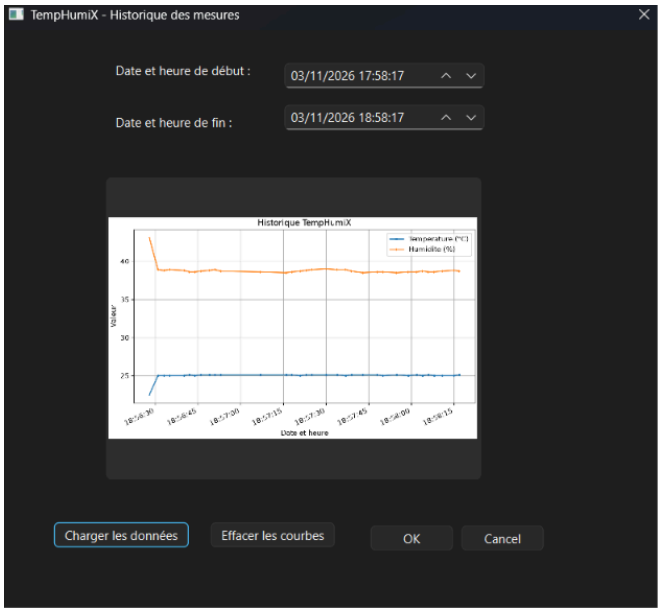


Figure 36 — Historique des mesures sur la dernière heure disponible.

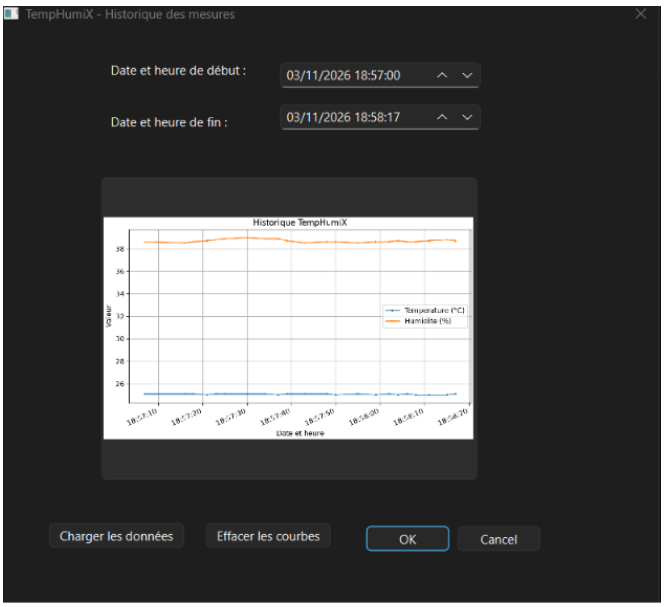


Figure 37 — Même historique après réduction de la période à 18 h 57 – 18 h 58 min 17 s.

La première courbe porte sur une heure ; la seconde montre un intervalle plus court (03/11/2026, de 18:57:00 à 18:58:17). Le graphique est alors recentré sur les valeurs proches de 25 °C et de 39 % : la comparaison illustre le filtrage par date à partir des mêmes données archivées.

3.9 Bilan de l'IHM

Ce TP m'a permis de relier la communication MQTT, une interface graphique et une base SQLite dans la même application. Les principales difficultés ont été la configuration des variables d'environnement dans plusieurs terminaux, mes premiers pas sur Qt Designer et le traitement des messages sans bloquer l'interface. Les vérifications sur l'Arduino, dans les consoles Python puis dans SQLite m'ont permis de valider chaque étape progressivement.

L'application permet de consulter les mesures, commander le capteur, conserver un historique et examiner les courbes sur une période choisie. À l'avenir, je pourrais améliorer la lisibilité des graphiques, ajouter une indication en cas de perte de connexion et prendre en charge plusieurs capteurs.

Conclusion générale

Les tests progressifs sous SimulIDE ont permis de valider séparément le clignotement des LED, le bargraphe, le capteur DHT22 et l'affichage LCD avant de construire le montage complet. La simulation a également permis de vérifier les échanges sur la liaison série et le passage à la minute suivante.

Sur la carte Arduino UNO R4 WiFi, la connexion au réseau, la synchronisation NTP et la communication MQTT ont ensuite été testées avec MQTTX et observées dans Wireshark. Les essais ont mis en évidence le fonctionnement des publications, des abonnements, des commandes du boîtier et des niveaux de qualité de service. L'optimisation des publications a limité les messages répétés lorsque les mesures ne changent pas.

Enfin, l'IHM Python développée avec PySide6 permet de recevoir les valeurs et d'envoyer les commandes tout en gardant la fenêtre utilisable grâce au thread MQTT. L'archivage SQLite et les courbes Matplotlib complètent la chaîne, de l'acquisition jusqu'à la consultation d'un historique filtré par dates.

Les principales difficultés ont concerné la configuration des outils de simulation, l'accès à la liaison série, les paramètres réseau et les premiers essais avec Qt Designer. La progression par petites vérifications a permis de les résoudre et de conserver des preuves du fonctionnement de chaque étape. Une évolution possible serait de rendre la supervision plus robuste en cas de perte de connexion et d'améliorer l'ergonomie de consultation des historiques.

Dépôt du projet : https://github.com/khadija-oua/projet-iot